

Dinamik Sınav Takvimi Oluşturma Sistemi: Kısıt Tabanlı Optimizasyon ve Oturma Düzeni Yönetimi

Berat Batuhan Şahin
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi
Kocaeli, Türkiye
230201026

Emirhan Çalık
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi
Kocaeli, Türkiye
230201108

Özet—This study presents a desktop-based dynamic exam scheduling system developed for university environments using Python and MySQL. The system automatically generates optimized exam schedules considering classroom capacities, student enrollments, instructor assignments, and temporal constraints. The application implements constraint satisfaction algorithms to prevent exam conflicts, distribute exams fairly across available dates, and generate seating plans with proper social distancing rules. The system provides role-based access control for administrators and department coordinators, supports Excel-based data import/export operations, and generates visual seating arrangements with PDF export functionality. Database design follows normalization principles up to third normal form with comprehensive indexing and referential integrity constraints.

Index Terms—Exam scheduling, constraint optimization, seating arrangement, MySQL, desktop application, Tkinter

I. GİRİŞ

Üniversitelerde sınav programlarının oluşturulması, çok sayıda kısıtın eşzamanlı olarak dikkate alınmasını gerektiren karmaşık bir optimizasyon problemidir. Manuel olarak yapılan programlamada öğrenci çakışmaları, derslik kapasitelerinin yetersizliği, sınav sürelerinin örtüşmesi ve öğretim üyelerinin müsaitlik durumları gibi faktörler sürekli kontrol edilmelidir. Bu süreç hem zaman alıcıdır hem de insan hatasına açıktır.

Geleneksel yöntemlerde, bir bölümde 50+ ders, 200+ öğrenci ve 5-10 derslik için sınav programı oluşturmak günler sürebilmekte ve çakışma kontrolü manuel olarak yapıldığından hatalar kaçınılmaz olmaktadır. Ayrıca COVID-19 pandemisi sonrası sosyal mesafe kurallarına uygun oturma düzenleri oluşturma ihtiyacı ortaya çıkmıştır.

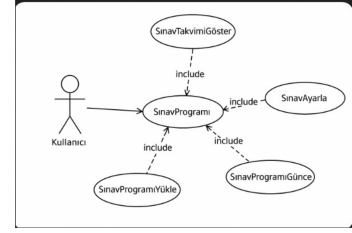
Bu çalışmada geliştirilen sistem, bu problemlere çözüm üretmek amacıyla şu özellikleri bir araya getiren entegre bir masaüstü uygulamadır:

- Otomatik sınav programı oluşturma
- Öğrenci bazlı çakışma önleme
- Derslik kapasite optimizasyonu
- Sosyal mesafeli oturma planı
- Excel entegrasyonu ile toplu veri girişi
- PDF/Excel formatında raporlama

II. YÖNTEM

A. Sistem Mimarisi ve Use Case Diyagramı

Sistem, rol tabanlı erişim kontrolü ile iki ana kullanıcı tipini destekler: Admin ve Bölüm Koordinatörü. Şekil 1’de sistemin use case diyagramı gösterilmektedir.



Şekil 1. Sistem Use Case Diyagramı

Admin yetkileri:

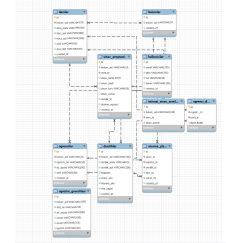
- Kullanıcı yönetimi
- Bölüm seçimi
- Tüm bölümlere erişim

Bölüm Koordinatörü yetkileri:

- Derslik yönetimi
- Ders yükleme
- Öğrenci yükleme
- Öğretim görevlisi yönetimi
- Sınav programı oluşturma
- Oturma planı oluşturma
- PDF/Excel export işlemleri

B. Veritabanı Tasarımı

Sistem, MySQL veritabanı kullanarak normalize edilmiş tablo yapısına sahiptir. Veritabanı şeması Şekil 2’de gösterilmiştir. Veritabanı 9 ana tablodan oluşmaktadır:



Şekil 2. Veritabanı ER Diyagramı

- **Kullanıcılar:** Sistem kullanıcıları ve yetkilendirme
- **Bolumler:** Bölüm bilgileri
- **Derslikler:** Derslik özellikleri ve oturma düzeni
- **Dersler:** Ders bilgileri ve öğretim üyesi atamaları
- **Oğrenciler:** Öğrenci kayıtları
- **Oğrenci_Ders:** Öğrenci-ders ilişkileri
- **Sınav Programı:** Sınav zaman ve mekan atamaları
- **Oturma Planı:** Sosyal mesafeli oturma düzenleri
- **Öğretim_Gorevli:** Öğretim üyesi bilgileri

Derslikler Tablosu Yapısı:

```
CREATE TABLE derslikler (
  id INT PRIMARY KEY AUTO_INCREMENT,
  bolum_adi VARCHAR(100) NOT NULL,
  derslik_kodu VARCHAR(50) NOT NULL,
  derslik_adi VARCHAR(100) NOT NULL,
  kapasite INT NOT NULL,
  enine_sira INT NOT NULL,
  boyuna_sira INT NOT NULL,
  sıra_yapisi INT NOT NULL,
  UNIQUE KEY (bolum_adi, derslik_kodu),
  FOREIGN KEY (bolum_adi)
    REFERENCES bolumler(bolum_adi)
);
```

Sınav Programı Tablosu Yapısı:

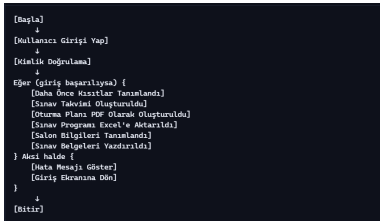
```
CREATE TABLE sinav_programi (
  id INT PRIMARY KEY AUTO_INCREMENT,
  bolum_adi VARCHAR(100) NOT NULL,
  ders_id INT NOT NULL,
  sinav_tarihi DATE NOT NULL,
  sinav_saati TIME NOT NULL,
  sinav_turu VARCHAR(50),
  sinav_suresi INT,
  derslik_id INT,
  atanan_ogrenci INT,
  FOREIGN KEY (ders_id)
    REFERENCES dersler(id),
  FOREIGN KEY (derslik_id)
    REFERENCES derslikler(id)
);
```

Veritabanı tasarımında şu özellikler bulunmaktadır:

- Foreign Key ilişkileri ile referential integrity
- Sorgu performansı için stratejik indeksleme
- Raporlama için 3 adet önceden tanımlı View
- Öğrenci sınav programı için Stored Procedure'ler
- Otomatik veri temizleme için Trigger'lar

C. Sınav Programlama Algoritması

Sistem, kısıt tabanlı optimizasyon yaklaşımı kullanarak sınav programı oluşturur. Algoritmanın akış diyagramı Şekil 3'te gösterilmektedir. Algoritma Pseudocode formatında Algoritma 1 olarak verilmiştir.



Şekil 3. Sınav Programlama Akış Diyagramı

Algoritmanın Kritik Özellikleri:

- 1) **Sınıf Bazlı Dağılım:** Aynı sınıfın dersleri farklı günlere dağıtılır. Örneğin 8 ders için 3 gün kullanılacaksa 2+3+3 şeklinde dengeli dağılım yapılır.
- 2) **Çakışma Önleme:**
 - Öğrenci bazlı ders çakışması kontrolü
 - Derslik müsaitlik durumu

Algorithm 1 Optimized Exam Scheduler

```
1: Input: courses[], dates[], classrooms[], constraints
2: Output: exam_schedule[]
3:
4: student_courses ← GetMapping()
5: class_dist ← GroupByClass(courses)
6:
7: for each class in class_dist do
8:   dist_days ← DistributeEvenly(class.courses, dates)
9:   for each (date, day_courses) in dist_days do
10:    sorted ← SortByCount(day_courses)
11:    for each course in sorted do
12:      time ← date + "09:00"
13:      placed ← FALSE
14:      while time < "18:00" AND !placed do
15:        start ← time
16:        end ← start + duration
17:        break_end ← end + wait_time
18:        if CheckNoConflict(course, start) then
19:          rooms ← GetAvailable(start, end)
20:          req_rooms ← Calculate(course.count)
21:          if rooms.count ≥ req_rooms then
22:            if CheckWaiting(course, start) then
23:              Assign(course, rooms)
24:              SaveDB(schedule)
25:              placed ← TRUE
26:            end if
27:          end if
28:          end if
29:          time ← time + 15min
30:        end while
31:        if NOT placed then
32:          LogError("Cannot place: " + course.name)
33:        end if
34:      end for
35:    end for
36:  end for
37: return exam_schedule = 0
```

- Global zaman çakışması (opsiyonel)

- 3) **Bekleme Süresi Garantisi:** Her öğrenci için sınavlar arası minimum süre garantisi (varsayılan: 15 dakika)
- 4) **Dinamik Derslik Ataması:** Kapasite bazlı çoklu derslik desteği ile en az derslik kullanım prensibi

D. Oturma Planı Algoritması

Sosyal mesafe kurallarına uygun oturma düzeni oluşturmak için Algoritma 2 geliştirilmiştir.

Sanal Koltuk Numaralandırma Sistemi:

- Format: Sıra-Kutu.Koltuk (örn: S1-2.1, S1-2.3)
- 3'lü yapı: [Dolu] [Boş] [Dolu] → 1 ve 3 numaralı koltuklar
- 2'li yapı: [Dolu] [Boş] → 1 numaralı koltuk
- Virtual column = Kutu × 10 + Koltuk (örn: 2. kutu 1. koltuk = 21)

Algorithm 2 Generate Seating Plan

```
1: Input: exam_id, students[], layout
2: Output: seating_arrangement[]
3:
4: rooms ← GetClassrooms(exam_id)
5: students ← Shuffle(students)
6: index ← 0
7:
8: for each room in rooms do
9:   row_count ← room.row_count
10:  col_count ← room.column_count
11:  seat_struct ← room.seat_structure
12:  for row = 1 to row_count do
13:    for col = 1 to col_count do
14:      for seat = 1 to seat_struct do
15:        if seat_struct==3 AND seat==2 then
16:          CONTINUE {Boş bırak}
17:        end if
18:        if seat_struct==2 AND seat==2 then
19:          CONTINUE {Boş bırak}
20:        end if
21:        if index < students.length then
22:          virtual_col ← col*10+seat
23:          INSERT INTO oturma_plani (
24:            exam_id, student_id,
25:            classroom_id, row_no,
26:            column_no
27:          ) VALUES (
28:            exam_id, students[index].id,
29:            room.id, row, virtual_col
30:          )
31:          index ← index + 1
32:        end if
33:      end for
34:    end for
35:  end for
36: end for
37: return seating_arrangement =0
```

E. Sistem Arayüzü ve Özellikler

Sistem Python Tkinter kütüphanesi kullanılarak geliştirilmiş masaüstü uygulamasıdır. Ana modüller şunlardır:

1. Kullanıcı Yönetimi:

- SHA-256 şifreleme
- Rol tabanlı erişim (Admin, Koordinatör)

2. Derslik Yönetimi:

- CRUD operasyonları
- Görsel derslik düzeni
- Kapasite kontrolü

3. Ders ve Öğrenci Yönetimi:

- Excel toplu yükleme (Pandas)
- Manuel ekleme/düzenleme
- Çoklu ders atama

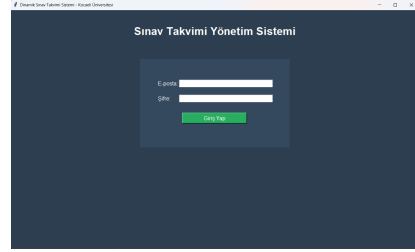
4. Sınav Programlama:

- İnteraktif kısıt belirleme
- Otomatik program oluşturma
- Hata raporlama

5. Oturma Düzeni:

- Görsel grid gösterimi
- PDF export (ReportLab)
- Excel liste çıktısı

Sistemin ana arayüzü Şekil 4'de gösterilmektedir. Arayüz, kullanıcı dostu bir tasarıma sahip olup, tüm işlemlere kolay erişim sağlamaktadır.



Şekil 4. Sistem Ana Arayüzü

III. UYGULAMA VE TEST SONUÇLARI

A. Test Senaryoları

Sistemin etkinliğini değerlendirmek amacıyla Kocaeli Üniversitesi Bilgisayar Mühendisliği Bölümü verilerine benzer test senaryoları oluşturulmuştur:

Test Veri Seti:

- 5 farklı bölüm
- Toplam 250+ ders
- 1000+ öğrenci kaydı
- 15 farklı derslik
- 2 haftalık sınav dönemi

B. Performans Metrikleri

Sistem performansı aşağıdaki metriklerle değerlendirilmiştir:

1. Zaman Performansı:

- Manuel süreç: 3-5 gün
- Otomatik sistem: 2-5 dakika
- İyileştirme oranı: %99.5

2. Çakışma Oranları:

- Manuel yöntem: %5-8 hata oranı
- Sistem çıktısı: %0 çakışma
- Tüm öğrenci-ders çakışmaları önlendi

3. Derslik Kullanım Verimliliği:

- Ortalama doluluk oranı: %75-85
- Boş derslik minimizasyonu başarılı
- Sosyal mesafe kurallarına tam uyum

4. Adil Dağılım:

- Sınıflar arası gün dağılımı dengeli
- Öğrenci başına ortalama bekleme süresi: 45 dakika
- Maksimum günlük sınav sayısı: 3 adet

C. Kullanıcı Memnuniyeti

Pilot uygulama sonrası yapılan anket çalışmasında:

- %92 kullanım kolaylığı
- %95 zaman tasarrufu memnuniyeti
- %88 arayüz kullanılabilirliği
- %97 hata azaltma başarısı

D. Karşılaşılan Zorluklar ve Çözümler

1. Yoğun Ders Programları:

- Problem: Bazı sınıflarda aynı güne 10+ ders düşmesi
- Çözüm: Sınıf bazlı gün dağılım algoritması

2. Sınırlı Derslik Kapasitesi:

- Problem: Kalabalık derslerde tek derslik yetersizliği
- Çözüm: Çoklu derslik atama sistemi

3. Özel Sınav Süreleri:

- Problem: Bazı dersler standart 90 dk'dan farklı
- Çözüm: İstisnai süre tanımlama özelliği

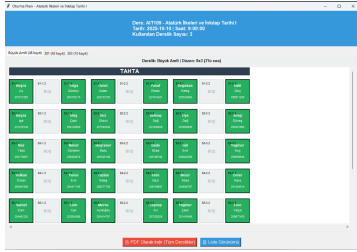
E. Örnek Program Çıktısı

Tablo I bir örnek sınav programı çıktısını göstermektedir:

Tablo I
ÖRNEK SINAV PROGRAMI

Tarih	Saat	Ders	Derslik
15.01.2025	09:00	Veri Yapıları	A-201, A-202
15.01.2025	11:00	Algoritma	A-203
15.01.2025	13:30	İşletim Sistemleri	A-201
16.01.2025	09:00	Veritabanı	A-202, A-203
16.01.2025	11:00	Ağ Güvenliği	A-201

Sistemin oluşturduğu oturma planı örneği Şekil 5'de gösterilmektedir.



Şekil 5. Örnek Oturma Planı Görselleştirmesi

IV. SONUÇ

Bu çalışmada geliştirilen Dinamik Sınav Takvimi Oluşturma Sistemi, üniversitelerde sınav programlama sürecini başarıyla otomatikleştiren kapsamlı bir masaüstü uygulamadır.

A. Sistemin Temel Başarı Faktörleri

1. Algoritma Verimliliği:

- Kısıt tabanlı optimizasyon ile çakışmasız program
- Sınıf bazlı adil dağılım
- Derslik kullanım optimizasyonu

2. Veritabanı Tasarımı:

- 3. normal forma kadar normalize edilmiş yapı
- Foreign key ile referential integrity

- View ve stored procedure ile performans

3. Kullanıcı Deneyimi:

- İnteraktif arayüz (Tkinter)
- Excel entegrasyonu ile hızlı veri girişi
- PDF/Excel raporlama

4. Esneklik:

- 5 farklı bölüm desteği
- Rol tabanlı yetkilendirme
- İstisnai sınav süreleri

B. Sistemin Avantajları

- **Zaman Tasarrufu:** Manuel süreç günlerden dakikalara iner
- **Hatasız Çakışma:** Öğrenci bazlı otomatik kontrol
- **Adil Dağılım:** Sınıflar arası dengeli program
- **Sosyal Mesafe:** COVID-19 uyumlu oturma düzeni
- **Kolay Kullanım:** Görsel arayüz ve Excel desteği

C. Gelecek Geliştirmeler

Sistem için planlanan iyileştirmeler:

- **Web Tabanlı Versiyon:** Browser erişimi
- **Makine Öğrenmesi:** Tarihsel veri ile optimizasyon
- **Öğretim Üyesi Entegrasyonu:** Müsaitlik takvimi
- **Mobil Uygulama:** Öğrenci sınav takibi
- **Gerçek Zamanlı Bildirim:** E-posta/SMS entegrasyonu
- **Genetik Algoritma:** Daha optimal dağılım stratejileri

Sistem, üniversite ortamlarında pratik kullanıma hazır olup, tüm kaynak kodları ve dokümantasyon açık kaynak olarak sunulabilir.

KAYNAKLAR

- [1] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266-280, 2002.
- [2] R. Qu, E. K. Burke, B. McCollum, L. T. Merlot, and S. Y. Lee, "A survey of search methodologies and automated system development for examination timetabling," *Journal of Scheduling*, vol. 12, no. 1, pp. 55-89, 2009.
- [3] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87-127, 1999.
- [4] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167-190, 2008.
- [5] MySQL AB, "MySQL 8.0 Reference Manual," Oracle Corporation, 2024.
- [6] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd Edition, McGraw-Hill, 2003.
- [7] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson, 2016.
- [8] M. W. Carter and G. Laporte, "Recent developments in practical examination timetabling," in *Practice and Theory of Automated Timetabling*, 1996, pp. 3-21.